SmartCourse AI — Sample Course Notes (1/3): Retrieval-Augmented Generation

This sample handout is original demonstration material bundled with the public repository. It is intended to test PDF ingestion and answers with source citations, and is not an official university lecture.

What is RAG? Retrieval-Augmented Generation combines information retrieval with language-model generation. First a knowledge base is segmented into short chunks. The text chunks are converted into numerical vectors called embeddings and stored in a vector database.

At question time, the same embedding model converts the user query to a vector. A similarity search locates the most relevant chunks. The application places those chunks in the language model prompt, enabling a grounded answer that points back to the document and page where evidence appeared.

Source attribution: Retrieval sources should name the document and page number. Cosine similarity is a signal of semantic relatedness, not a guarantee of factual truth. If the document contains no relevant answer, the application must acknowledge the gap rather than invent citations.

Chunking: Overlapping chunks can help preserve information near chunk boundaries, but duplicate or irrelevant context can also crowd out useful evidence. Evaluate retrieval using representative questions.

SmartCourse AI — Sample Course Notes (2/3): Routing and Tools

Agent Router: Before generation, SmartCourse AI selects one of three paths for each user request. RAG serves questions about course documents. TOOL executes approved Python functions for structured tasks. DIRECT_LLM handles general questions that do not require documents or calculations.

Grade Calculator: Weighted final grades must be computed in Python rather than left to language-model arithmetic. For an exam grade of 82 weighted at 70 percent and a project grade of 95 weighted at 30 percent, the result is (82 × 0.70) + (95 × 0.30) = 85.9. Weights must sum to 100 percent and grades must be within the accepted range.

Course Lookup: Structured lookups can retrieve assignment dates and grade weights only when records are verified. If an official deadline has not been supplied, the correct behavior is to explain that no verified information is available.

Tool safety: A tool-use schema describes the available function and valid arguments. The server validates arguments, invokes only allowlisted functions, stores execution logs, and provides tool output back to the language model for a readable final response.

SmartCourse AI — Sample Course Notes (3/3): Anthropic Python SDK

How do I authenticate with the Anthropic Python SDK? Install the SDK in a virtual environment using `pip install anthropic`. Obtain an API key from the Anthropic developer console and store it in the environment variable `ANTHROPIC_API_KEY`. Do not embed the API key in source code or commit it to a public repository.

Creating the client: After setting the environment variable, Python code can call `import anthropic` and then `client = anthropic.Anthropic()`. The SDK reads credentials from the process environment. A local `.env` file can be loaded by `python-dotenv` during development; it must be ignored by Git.

Sending messages: A typical SDK request supplies a model identifier, token limit, and a list of user and assistant messages. Responses contain model-generated content. Respect timeout and retry limits and handle provider failures without leaking credentials or internal exceptions.

Security precaution: Protect API keys as secrets. If a key is exposed, revoke it and generate a new key. Validate user input, keep model calls on the server, and never expose database credentials to frontend JavaScript.